Task 3 – Simulating Human Knowledge Gaps: LLMs, Theory of Mind, and Science QA

Large Language Models are increasingly used not just to answer questions, but to *simulate other agents,* students, customers or naive users for purposes like tutoring system design, dataset augmentation, and behavioral research. Doing this well requires a form of Theory of Mind (ToM): the model must reason about what *another agent* believes or doesn't believe, and predict that agent's behavior accordingly, not simply produce the correct answer itself.

In this task, you will use an LLM (via LangChain) to (1) decompose science questions into their underlying prerequisite concepts, and (2) generate simulated answers from the perspective of a person who lacks knowledge of some of those concepts. You will then analyze where and how the model's simulations succeed or fail, including cases where the model "leaks" knowledge it was told to suppress.

Dataset: OpenBookQA: a multiple-choice science question dataset (originally from Allen AI), where each question is paired with a gold (correct) answer and three distractor options, designed to require both core scientific facts and broader common-sense knowledge to answer correctly. The dataset is available via the Hugging Face datasets library (load_dataset("allenai/openbookqa", "main")) or as raw JSONL files from the OpenBookQA GitHub repo. Download and load it yourself; document its structure (fields, number of questions, format of choices/answers, train/validation/test splits) before proceeding.

1. LLM & LangChain Fundamentals (22 Points)

1.0 API Setup

For this entire assignment, you will use Google AI Studio's free tier API (https://aistudio.google.com). This gives free, rate-limited access to Google's models via an API key.

Model selection: use this priority order, falling back only if you hit rate limits or quota errors with the higher option:

1. Gemma 4 31B
2. Gemma 4 26B

3. Gemini 3.1 Flash-Lite

Document in your notebook which model(s) you ended up using and why (e.g., "Used Gemma 4 31B for Sections 1–3, switched to Gemini 3.1 Flash-Lite for Section 4 due to rate limiting"). If you hit rate limits frequently: you are allowed to create multiple Google accounts and obtain multiple API keys, then rotate between them (e.g., a simple wrapper that switches keys when one hits a rate-limit error). This is optional but recommended if you find generation steps later in the assignment slow. Document if you do this.

1.1 Setup & Basic Calls (4 pts)

1. Obtain an API key from Google AI Studio. Set it as an environment variable (e.g., GOOGLE_API_KEY).
2. Configure LangChain to use your selected model (start with Gemma 4 31B) via langchain-google-genai.
3. Send a basic prompt and display the raw response.
4. Briefly document: which model(s) you used, any rate limit/quota errors you encountered, and how you handled them (including whether you used multiple API keys).

1.2 Prompt Templates (5 pts)

1. Define a PromptTemplate for a simple question-answering task (input: question + choices, output: free-text answer).
2. Run this template on 3 sample questions from your loaded dataset and display the outputs.

1.3 Structured Output Parsing (5 pts)

1. Define a Pydantic schema for a structured response containing at least: answer (the selected choice), reasoning (free text), and confidence (numeric, 0–1).
2. Use a structured output parser (e.g., PydanticOutputParser or with_structured_output) to get responses in this format for the same 3 questions.

3. Trigger and document at least one case where the model's output failed to parse correctly. Show how you handled it (e.g., retry, OutputFixingParser, or a manual repair step).

1.4 Dataset Enrichment: Adding Subject & Difficulty Labels (8 pts)

OpenBookQA does not include subject area or difficulty annotations. In this section, you'll use the LLM and structured output parsing to generate these labels yourselves – a common real-world use of LLMs for dataset annotation/enrichment. The 100 questions you select here will be the working dataset for the rest of this assignment. all later sections (including Section 3's question selection) draw from this set, not the full OpenBookQA dataset.

1. Randomly sample 100 questions from OpenBookQA (set a random seed and document it for reproducibility).
2. Define a Pydantic schema for an annotation response containing at least:

   subject_area: one of a fixed set of categories you define (e.g., physics, biology, chemistry, earth_science, astronomy, other). pick a small, fixed list of 4–6 categories *before* running this, so labels are consistent across questions.

   difficulty: an integer 1–5 (1 = basic recall, 5 = requires multi-step reasoning or combining multiple concepts), with a short rubric you define for what each level means.

   reasoning (brief justification for the labels. useful for spot-checking).
3. For each of the 100 questions, prompt the LLM (using your structured output parser from 1.3) with the question text and choices, and collect subject_area, difficulty, and reasoning.
4. Save the enriched dataset as enriched_questions.csv (original fields + subject_area, difficulty, reasoning).
5. Spot-check 10 random rows: do you agree with the assigned subject_area and difficulty? Report your agreement rate with brief notes on disagreements.
6. Plot the distribution of subject_area and difficulty across your 100 questions (two simple bar charts).

2. Theory of Mind Background (17 Points)

2.1 What is Theory of Mind? (4 pts)

In a few sentences, explain what Theory of Mind (ToM) is, including the distinction between first-order ToM ("what does person A believe?") and second-order ToM ("what does A believe B believes?"). Briefly mention the false-belief task paradigm (e.g., the Sally-Anne test) as a classic way of testing ToM.

2.2 Hands-On: Testing a Classic False-Belief Task (7 pts)

1. Write out 2-3 classic false-belief scenarios as prompts (e.g., a Sally-Anne-style scenario, or adapt one from a benchmark like ToMi). Use your structured output parser from Section 1.3 to get the model's answer + reasoning.
2. Does your model pass these? Show the actual outputs.
3. If it passes, does the reasoning suggest genuine perspective-tracking, or could it be pattern-matching on a familiar template? Try a slightly modified/novel version of one scenario and see if performance holds; report what you find.

2.3 Connecting ToM to This Task (6 pts)

In 2-3 paragraphs, explain:

- Why generating "what would a person who doesn't know X answer" is a ToM-relevant task, distinct from simply answering the question correctly.
- What it would mean for the model to "fail" at this task? distinguish at least two different failure modes (think about what could go wrong, even if you haven't run experiments yet. you'll revisit this in Section 5).

3. Concept Extraction (22 Points)

This is the first agentic step: you will use the LLM itself to decompose questions into prerequisite knowledge.

3.1 Pilot Sample Selection (3 pts)

From your enriched_questions.csv (Section 1.4), select 8 questions spanning at least 3 different subject_area values and at least 3 different difficulty levels (your 8 don't need to

cover every combination, but shouldn't all cluster at one difficulty level — this matters for later analysis). Document your selection (question IDs, text, gold answers, distractors, subject area, difficulty) in a table. This pilot set is used to design and validate your extraction prompt before scaling to the full 100.

3.2 Concept Extraction Prompting (8 pts)

1. Design a prompt (with structured output, building on Section 1.3) that asks the LLM to extract exactly 3 prerequisite concepts required to answer a given question, ordered from hardest/most advanced (concept 1) to easiest/most basic (concept 3).
    - A "concept" should be a generalizable piece of knowledge or skill (e.g., "thermal conductivity varies by material"), not a fact specific to this question's exact wording, and should not directly restate or give away the answer choice.
2. Define a Pydantic schema for this output, e.g., concepts: List[ConceptItem] where each ConceptItem has name, description, and difficulty_rank.
3. Few-shot prompting: you may include 1-2 worked examples directly in your prompt (i.e., a question + its ideal concept decomposition, written by you) to steer the model toward the granularity and style you want. This is optional but often improves consistency. if you use it, show your few-shot example(s) and briefly note whether it changed the outputs compared to a zero-shot version.
4. Run this extraction for all 8 pilot questions and save results to concepts_pilot.csv (question_id, concept_1, concept_2, concept_3, with descriptions).

3.3 Calibration / Sanity Check (6 pts)

1. Pick 2 of your 8 pilot questions (ideally at different difficulty levels) and manually write out what you (as the human annotator) would consider the "ideal" 3-concept decomposition, with brief justification for the hardest→easiest ordering.
2. Compare your manual decomposition to the model's output for these 2 questions:
    - Is the hardest→easiest ordering sensible, or did it sometimes invert difficulty?
    - Are the 3 concepts meaningfully distinct, or do any overlap/restate each other at different granularity?

3. If you spot systematic issues (redundant concepts, gives away the answer, bad ordering) across your pilot questions, revise your prompt and re-run on the pilot before proceeding. Document what you changed and why.

3.4 Scaling to the Full Dataset (5 pts)

1. Using your validated prompt from 3.2/3.3, run concept extraction on all 100 questions from enriched_questions.csv.
2. Save results to concepts_all.csv (question_id, concept_1, concept_2, concept_3, with descriptions).
3. Spot-check 5 additional random rows (outside your original pilot) and report whether the same quality holds at scale, or whether you notice new failure patterns that didn't appear in your 8-question pilot.

4. Generating Simulated Naive-Human Answers (28 Points)

4.1 Prompt Design: Two Strategies (6 pts)

For each question and each knowledge level k = 1, 2, 3 (meaning the simulated person does not know concepts 1 through k, using your hardest→easiest ordering), design prompts implementing two strategies:

- Strategy A. Persona Suppression: "You are a person who has never learned about [concept list]. Answer the following question as best you can."
- Strategy B. Reasoning Trace: "Simulate the step-by-step reasoning of a person who knows everything relevant to this topic except [concept list], then give their final answer."

Show one fully worked example prompt (any question, any k) for each strategy.

4.2 Pilot Generation (8 pts)

1. Using your 8 pilot questions from Section 3, for all combinations of k=1,2,3 and both strategies (8 × 3 × 2 = 48 generations), call the LLM using your structured output parser from Section 1.3 (reuse the answer/reasoning/confidence schema).
2. Save results to simulated_responses_pilot.csv with columns: question_id, k, strategy, simulated_answer, reasoning, confidence, raw_output.

3. Display a few example rows, including at least one where the simulated answer differs from the gold answer.

4.3 Expected Naive Answers: Pilot (7 pts)

For each (question, k) pair in your pilot set, you need a reference point representing what a real naive person might plausibly answer.

1. For the 2 questions you manually annotated in 3.3, write out for each k=1,2,3 what you'd expect a naive person to answer, and which distractor (if any) that corresponds to. Justify your reasoning.
2. For the remaining 6 pilot questions, use the LLM (structured output) to propose a "missing concept k → likely distractor" mapping with justification. Review all 6 and either accept or correct each. document any corrections and why.
3. Save to expected_naive_answers_pilot.csv (question_id, k, expected_answer, justification, source: manual/llm-accepted/llm-corrected).

4.4 Strategy Selection & Scaling (7 pts)

1. Based on your pilot results (4.2-4.3), pick one strategy (A or B) to use going forward. Justify your choice using concrete evidence from the pilot. e.g., agreement with expected naive answers, qualitative reasoning quality, or anything you noticed in 4.2's example outputs.
2. Select 30 additional questions (random sample) from the remaining 92 in enriched_questions.csv, spanning a range of subject_area and difficulty values.
3. For these 30 questions × k=1,2,3 using your chosen strategy only (90 generations), repeat the generation process from 4.2. Save to simulated_responses_scaled.csv.
4. Generate expected naive answers for these 30 questions using the LLM-assisted approach from 4.3.2 (no manual annotation required at this scale), with a spot-check on 5 random rows, report your agreement rate. Save to expected_naive_answers_scaled.csv.

5. Failure Analysis (16 Points)

5.1 Strategy Comparison (Pilot Only) (5 pts)

Using your pilot results (8 questions × 3 k × 2 strategies), plot, for each strategy, the % of simulated answers matching the gold answer and the % matching the expected naive answer, as a function of k. Does this pattern support the strategy choice you made in 4.4? If the pilot data is ambiguous or points the other way, say so honestly. you don't need to redo Section 4, just reflect on what you'd do differently.

5.2 Knowledge Leakage (Combined Dataset) (6 pts)

"Leakage" is when the model's reasoning still relies on a concept it was told the simulated person doesn't know. e.g., if concept 1 is "thermal conductivity varies by material" and the k=1 reasoning still argues from that fact, the model's own knowledge has leaked into a persona that's supposed to lack it. Define a concrete criterion for detecting this in the reasoning text, then measure the leakage rate as a function of k. Does it increase or decrease as more concepts become "forbidden"? Break this down further by difficulty and subject_area: do harder questions or certain subjects leak more, and why might that be? Show 3 annotated examples with the leaked concept highlighted in the reasoning text.

5.3 Failure Mode Classification (Combined Dataset) (5 pts)

For every (question, k) pair, classify the outcome as one of: matches expected naive (success), matches gold despite missing concepts (over-suppression / leakage-driven), or matches neither (off-distribution). Plot a stacked bar chart of these three categories across k=1,2,3, and again broken down by difficulty. Is "matches neither" more common at high or low difficulty, and what might explain that? Finally, pick 2 "matches neither" cases and hypothesize what went wrong: bad concept extraction, an ambiguous expected-answer mapping, or model confusion.

6. Comparative Summary (5 Points)

6.1 Summary Table (2 pts)

Using your pilot data (8 questions, both strategies): Rows = Strategy A and Strategy B, each at k=1,2,3. Columns: % match gold, % match expected naive, % leakage, % off-distribution.

6.2 Confidence Calibration (2 pts)

Using your combined dataset (chosen strategy): plot the model's reported confidence against whether the simulated answer matched the expected naive answer. Is the model's confidence calibrated to this task (predicting a naive person), or does it seem to just reflect the model's confidence in the true answer regardless of the persona? Give evidence for your conclusion.

6.3 Best Configuration (1 pts)

Based on 6.1 (pilot), which (strategy, k) combination produced the most "human-like naive" outputs, and does this match the strategy you chose to scale with in 4.4? If your combined-dataset results (Section 5) tell a different story at scale, note that too.

Final Discussion

Think through these. you don't need to submit written answers, but be prepared to discuss.

1. Is "pretend you don't know X" fundamentally different from "predict what a naive person would say"? Which is the model actually doing, based on your leakage analysis?
2. If concept extraction (Section 3) had been done by a different LLM, or by a human expert, how much do you think your downstream results would change? Where is your pipeline most sensitive to errors in early steps?
3. How would this task change if you had access to real survey data from intro-level students instead of heuristic "expected naive" answers?
4. What are the risks of using LLM-simulated "naive learner" data to train educational tools (e.g., adaptive tutors)? Where might simulated data mislead more than it helps?